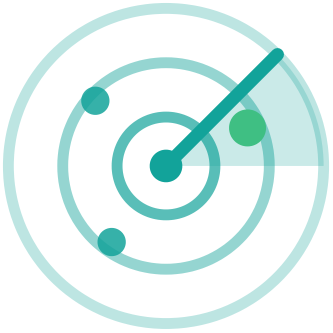


WhatsOnMyNet

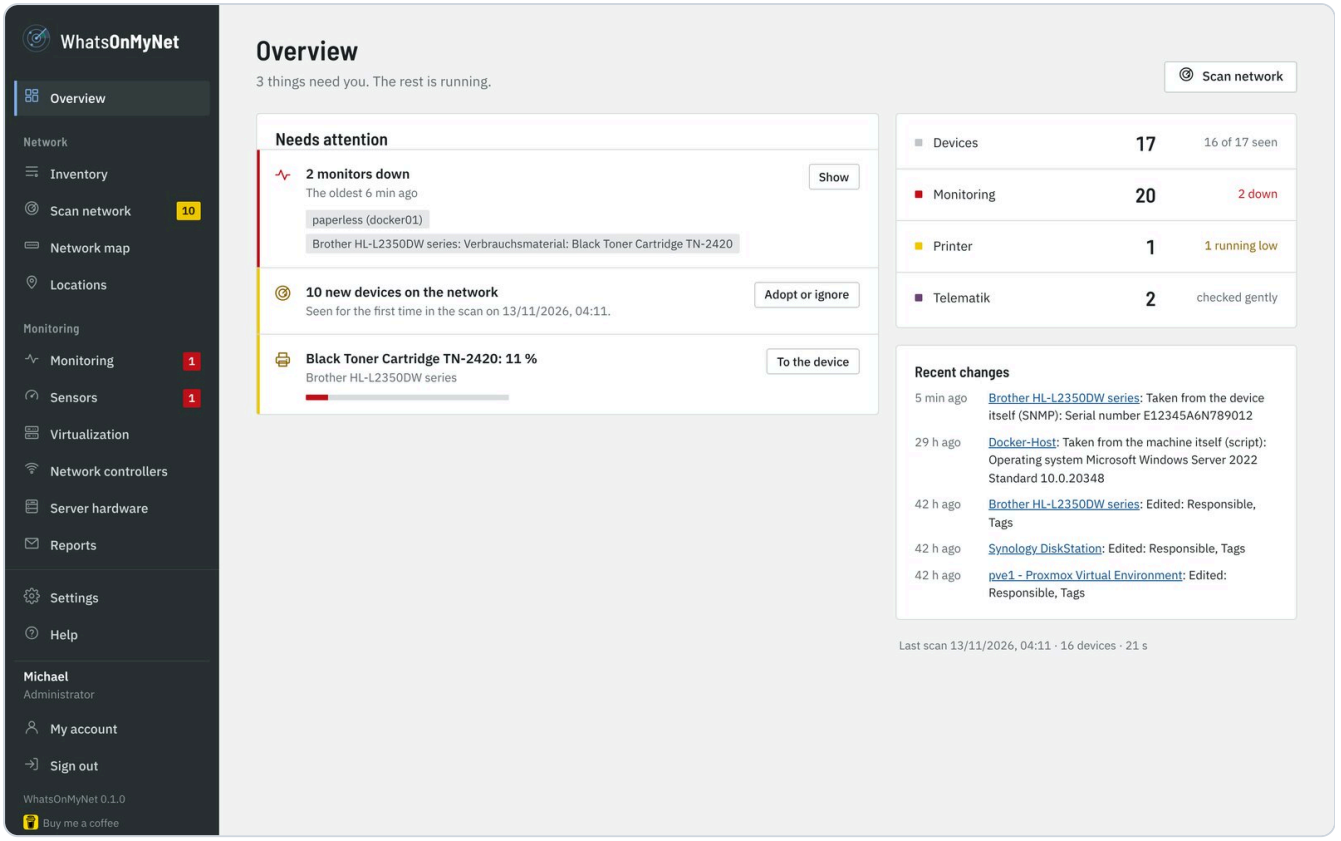
Manual - Network inventory and monitoring

2026-11-14

Contents

1. Installation
2. First steps
3. Inventory
4. Nightly light scan
5. Monitoring
6. Printers, switches and NAS
7. Sensors
8. Network drives (SMB)
9. Proxmox VE and Backup Server
10. More hypervisors: VMware, XCP-ng, Hyper-V, Nutanix, oVirt, KVM
11. Network controllers: UniFi, Omada, MikroTik
12. Server hardware: iDRAC, iLO, XClarity, Supermicro (Redfish)
13. Windows and Linux machines by script (without agent)
14. Reports
15. Users and rights
16. Sign-in with Active Directory
17. Scan agent for branch offices
18. Failover: master and standby
19. Backup and updates
20. Troubleshooting

WhatsOnMyNet finds what runs on your network and keeps only what you adopt in the inventory. It also monitors devices, services and websites, reads printers, switches, NAS and hypervisors (Proxmox, VMware, Hyper-V …), and reports by e-mail, messenger, push to the phone or webhook when something is wrong.



Overview

Installation

WhatsOnMyNet runs as a Docker container with host networking - only this way the scan sees the MAC addresses of the devices.

1. Put `docker-compose.yml` and `env.example` from the release into a folder, rename `env.example` to `.env`.
2. Adjust ports (`WOMN_HTTPS_PORT`, default 8743), time zone (`TZ`) and owner of the data (`PUID` / `PGID`) in `.env` if needed.
3. `docker compose up -d` - or create the stack in Dockhand.
4. Get the **setup code** from the log (`docker compose logs whatsonmynet`) and open `https://<server>:8743/setup`. There you create the first administrator.

The interface always runs over HTTPS. On first start WhatsOnMyNet creates a self-signed certificate; upload your own (PEM or PFX) under *Settings → HTTPS certificate*.

Ping: the container runs without root rights. Ping works if the host allows ICMP for normal users (`net.ipv4.ping_group_range`, default on Debian/Ubuntu/Proxmox). Without ping the scan still finds devices via ARP and TCP.

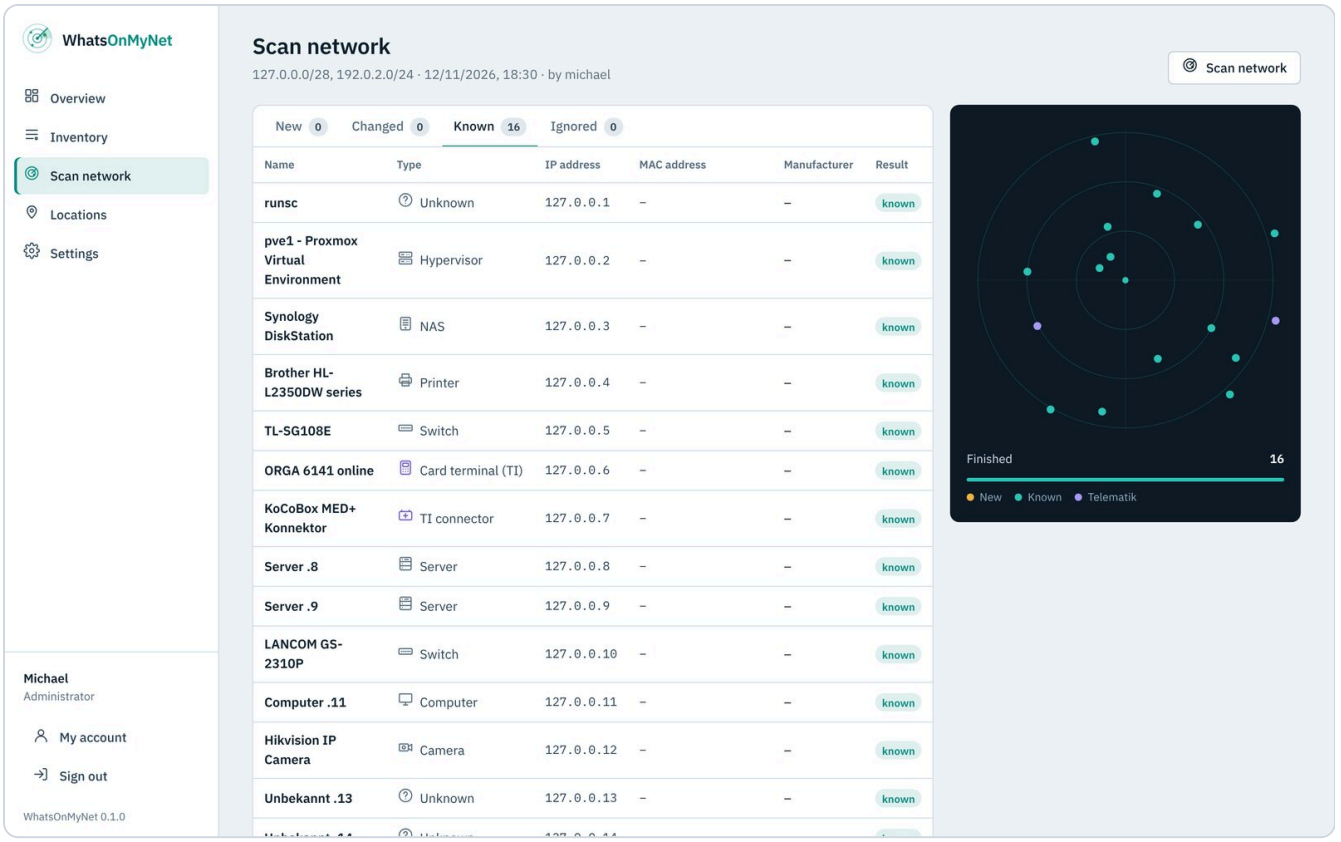
Allow ping in a Proxmox LXC (it is usually off there - `cat /proc/sys/net/ipv4/ping_group_range` shows `1 0`). Inside the LXC as root, not on the Proxmox host:

```
echo 'net.ipv4.ping_group_range = 0 65535' > /etc/sysctl.d/99-ping.conf
sysctl -p /etc/sysctl.d/99-ping.conf
docker compose restart
```

The setting survives a restart and needs no extra rights for the container. Higher than `65535` does not work in an unprivileged LXC ("Invalid argument"). It cannot go into `docker-compose.yml`: Docker does not allow `sysctls` in the host network.

First steps

1. **Add a location** (*Locations*): name and subnets, e.g. "Main building" with `192.168.10.0/24`.
2. **Scan the network** (*Scan network*): tick the location, "Start scan". The radar shows the devices found; the scan is gentle (ARP, 26 typical ports, SNMP read-only).
3. **Adopt**: in the result under "New" check type and name, tick devices, "Adopt into inventory". What you do not want in the inventory, "Ignore". Ignored by mistake? Tick it in the "Ignored" tab and choose "Stop ignoring" (it is listed under "New" again) or "Adopt into inventory" right away. The full list is under *Settings → Ignored devices*. Adopt and ignore work only from the newest scan of a subnet - once a newer scan has covered the subnet, an older scan stays readable, adopting or ignoring happens in the newer one. "Stop ignoring" works in the older scan too.



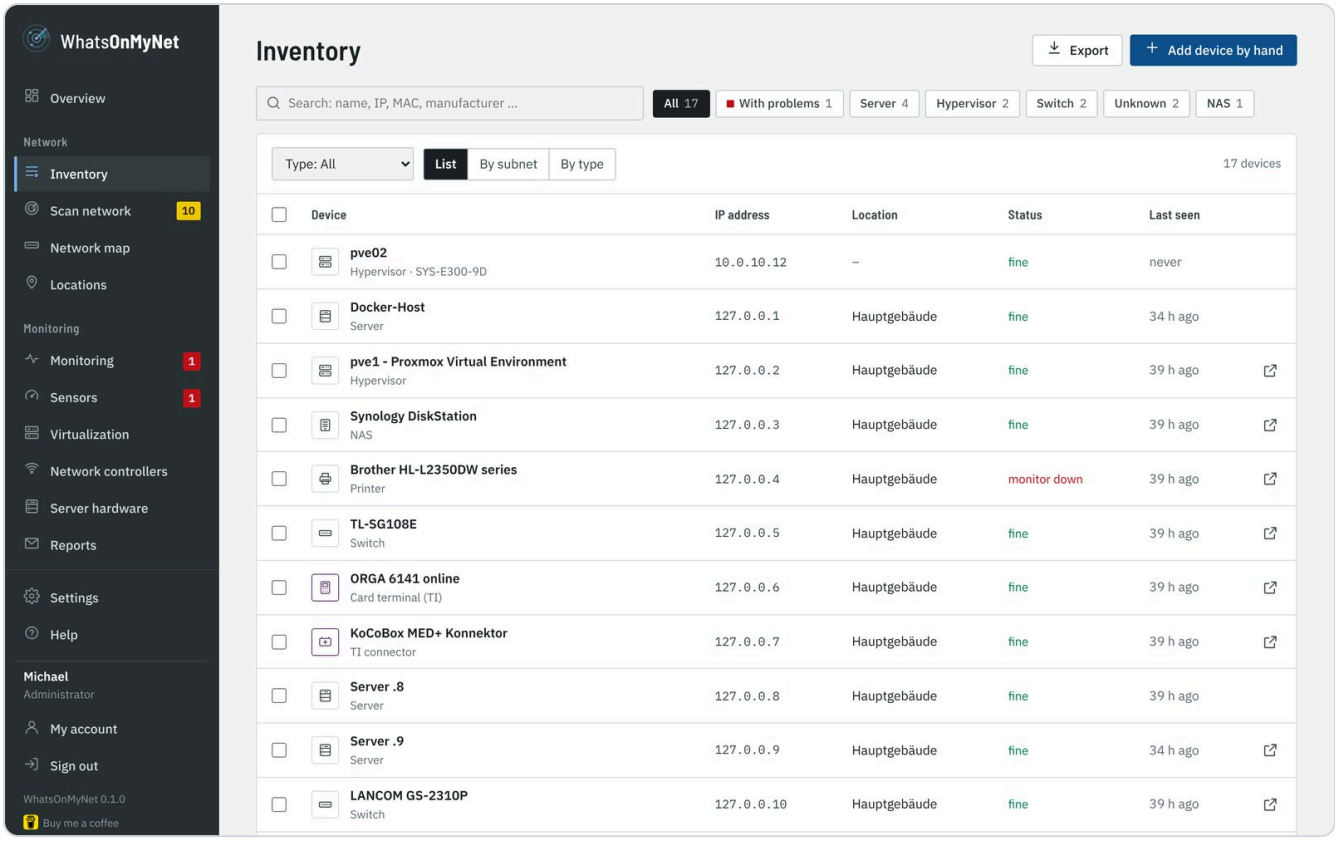
Scan with radar

The MAC address is the identity of a device: if it gets a new IP, the next scan takes it over automatically and records it in the history. A host name you typed by hand stays; the scan only fills the host name while the field is empty or the name came from the scan. If you empty the field, the scan fills it again.

In the inventory already? When the scan finds a device that is probably in the inventory already - same host name (e.g. the second network card of a NAS) or same IP with another MAC (VM, exchanged device) -, the list shows a tip with the existing device. Adopting skips it. "Assign to this device" takes over the new MAC for the same IP, or puts the second network card on the ignore list with a note; "Add as new device anyway" adds it after all. When devices are in the inventory twice already (same host name), the inventory shows a note with "Merge" at the top.

Nothing watched twice: WhatsOnMyNet does not create a second monitor or sensor for something watched already - it names the existing one. Host name and IP of the same device count as the same target; several services of one server (web interface, SSH, database ...) are of course fine. Windows and Linux machines without script show the tip "Monitor with script" on the device page.

Inventory



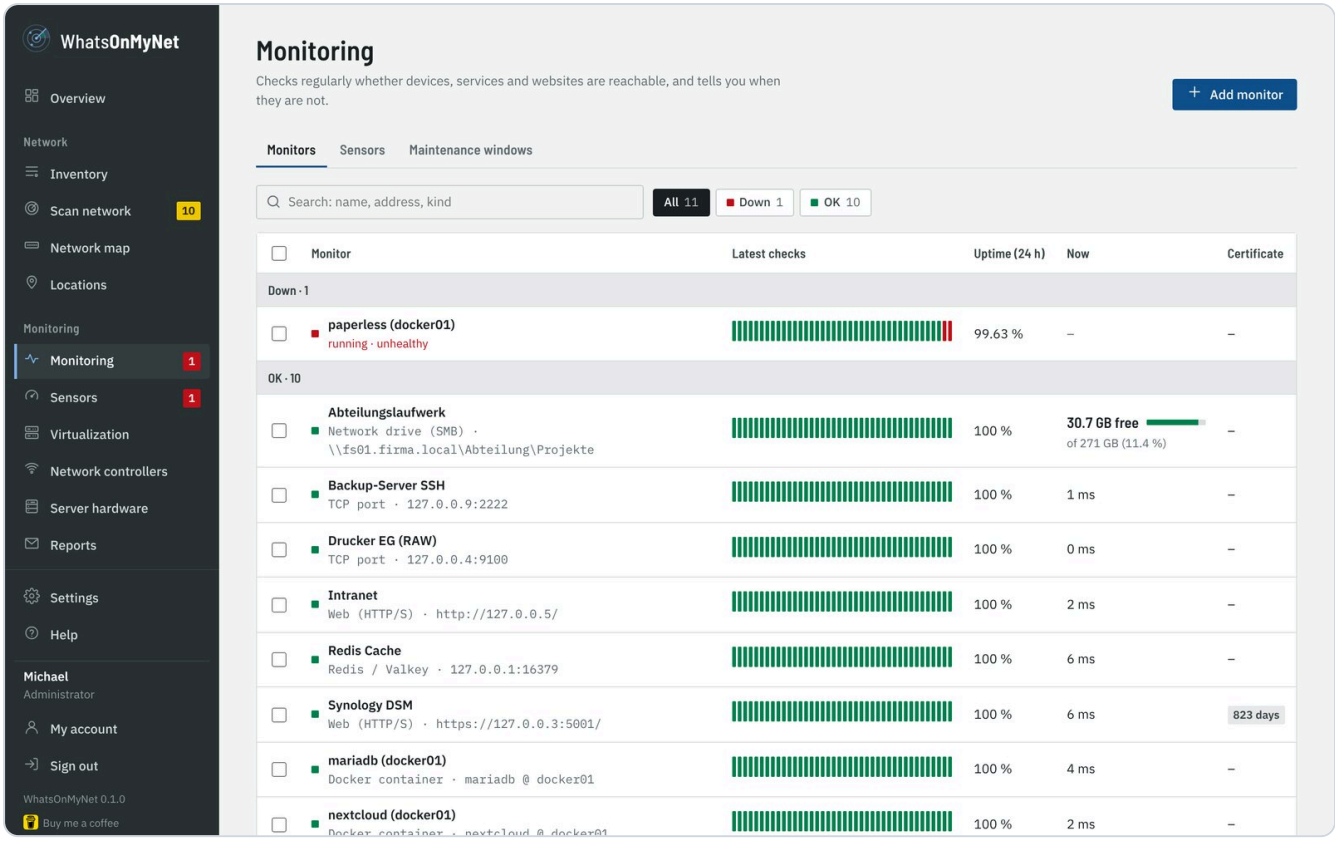
Inventory

- **Search and filter** by name, IP, MAC, manufacturer, type, location or tag; export as CSV or JSON.
- **Device page**: all details, purchase and warranty dates, notes, quick access to the web interface, what the scan detected, history of all changes.
- **Suggestions while typing**: manufacturer, model, operating system, owner, supplier and tags suggest what is already in the inventory. A known model fills in manufacturer and type.
- **Multiple selection**: tick devices and edit them together (type, location, manufacturer, model, dates, tags) or delete them.
- **Deep check**: checks one device with a longer port list. Telematik devices (connector, card terminal) are always checked gently. For a device not yet recognised, the deep check first asks port 4742 (SICCT) alone - if a card terminal answers, the gentle profile stays.

Nightly light scan

Every night (default 02:30) WhatsOnMyNet only looks for devices in all subnets of your locations: ARP, NetBIOS and - if allowed - ping; in routed subnets also a few TCP connection attempts (ports 443, 80, 22, 445, 3389) and the ARP table of the routers via SNMP. No port check of the devices, no web requests. Known devices get their new IP; a device showing up for the very first time is reported once. Settings, time and "Run now" are under *Settings → Scan*.

Monitoring



Monitoring

Watched right away: when you adopt devices from the scan into the inventory (or add one by hand), each gets a monitor at once: its web interface, else ping or an open port. As soon as the device delivers values, a few important sensors follow:

- **Printer:** every toner cartridge and faults
- **NAS:** volumes, RAID, health (system, fans, power supply) and temperature
- **UPS:** mains/battery, charge and runtime
- **Windows and Linux machines with the script:** online, CPU, RAM, system drive, highest temperature and failed services
- **Hosts under Virtualization:** nodes, storage and backups; for Backup Server datastores and tasks; for controllers the devices; for servers with a management chip the health

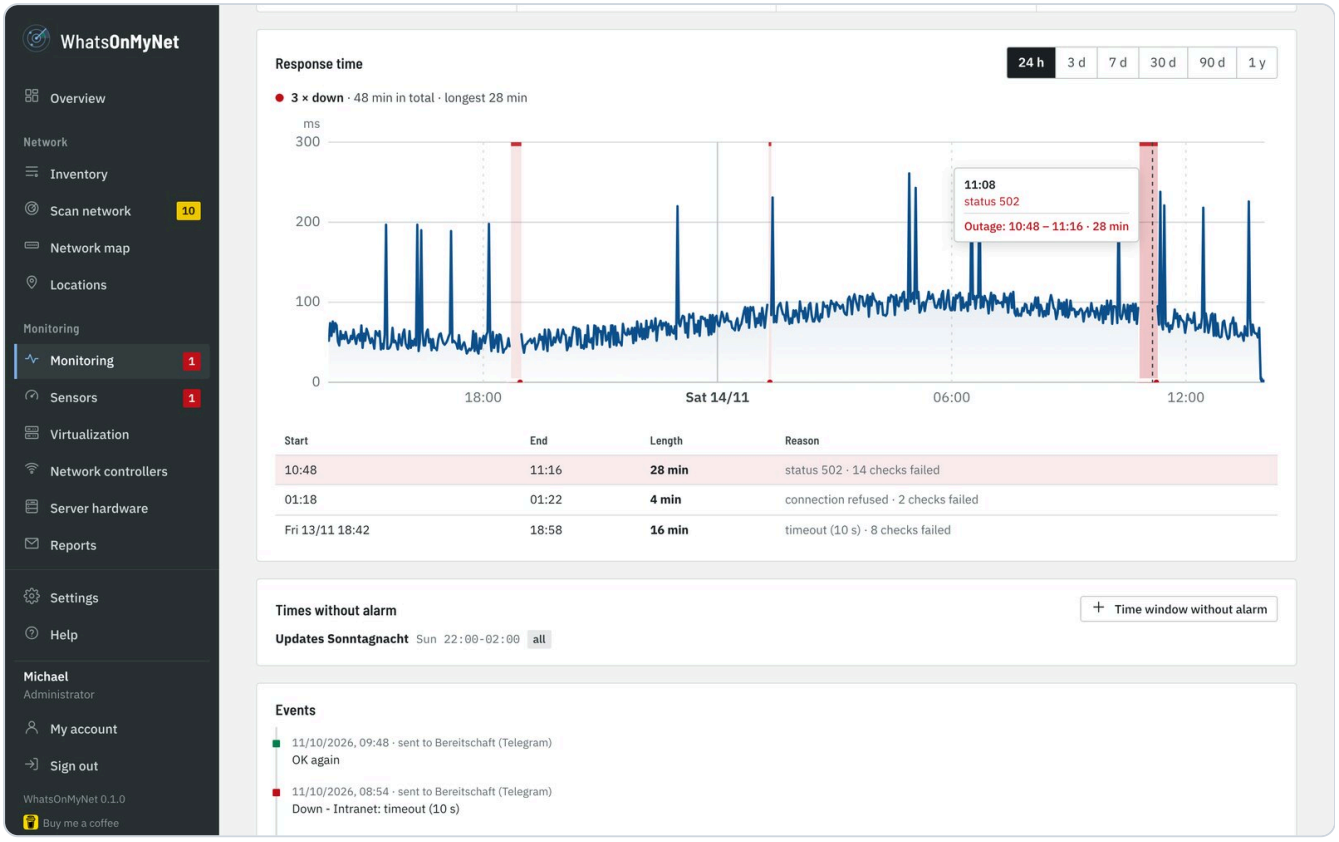
This applies to devices and hosts added from now on, and once each: if you delete one of these sensors, it does not come back. Switch it off under *Settings → Notifications → Watch new devices by themselves*.

A monitor checks regularly (20 seconds to 24 hours) and reports only after several failures in a row. Kinds:

- **Web (HTTP/S)** with status codes, keyword and certificate expiry (warning e.g. 30, 14 and 7 days before)
- **TCP port** and **ping**

- **Docker container** (through a socket proxy, *Settings → Docker*): running, restarting all the time, health check "unhealthy"
- **Databases**: MariaDB/MySQL, PostgreSQL, Redis/Valkey - sign-in and one harmless query
- **Network drive (SMB)**: Windows shares and NAS - see below
- **Sensor** (value of a device, a machine with script, a hypervisor or controller) and **SNMP value (OID)** - see below

On the device page "Monitor" creates a fitting monitor. If the IP of a device changes, its monitors follow. Exception: a monitor with a stored password (database, network share) is paused, because the password was entered for the old address and is never sent to another one unasked. It shows "Device has a new IP - please enter the password again", its notification channels get a message, and the edit dialog already has the new address filled in: saved with the password, it runs again. When the device returns to the old address, it runs on by itself. If you rename the device, its monitors and sensors take the new name too, so an alarm names it as it is called now (a monitor with a name of your own without the device name stays as it is).



Monitor with history

Notifications (*Settings → Notifications → Add channel*): choose the service, follow the short guide in the form, save and check with **Test message**.

- **Common:** e-mail (through the SMTP server), Telegram bot, webhook with your own JSON template (e.g. for n8n or a script)
- **Chat and messengers:** Discord, Slack, Microsoft Teams (Workflows), Google Chat, Mattermost, Rocket.Chat, Matrix/Element, Signal (through signal-cli-rest-api), WhatsApp (CallMeBot, to your own number)

- **Push to the phone:** ntfy, Gotify, Pushover, Pushbullet, Home Assistant (companion app)
- **Everything else:** Apprise - a bridge to more than 100 further services (SMS, Threema ...)

Tokens, keys and webhook addresses are stored encrypted and never shown again. Outages arrive with high priority on push services.

Maintenance windows (*Monitoring → Monitoring → Maintenance windows*): once or on certain weekdays from-to (also over midnight). A window is for the ticked monitors and sensors - the choice is grouped by device or hypervisor, with search, a tick at the device takes all of it. Nothing ticked = everything. Quicker for a single one: on the page of the monitor or sensor under *Times without alarm* → **Time window without alarm** (preset: daily 20-23 h, e.g. for the time of the backup in which an SSD gets warm). Checks and history go on in a maintenance window, only the messages are silent. The status "maintenance" is shown from the start of the window (not only after the next check) and goes with its end.

What an alarm says: web page, port and ping report "is down". A measured value reports "limit exceeded - 83.9 °C (≥ 80)" and later "back in the normal range", a state (service, VM, disk, container ...) "fault" and "fine again", a sensor without current values "no current values".

Changing several monitors or sensors at once

In the *Monitors* and *Sensors* lists every row has a box; the box in the table header takes all rows shown right now (search and filter apply). A bar above shows the number and the buttons **Pause**, **Resume**, **Delete** (asks first) and **Edit together**. The dialog holds the fields of *Advanced*: check every, alarm after ... failed checks, time limit, repeat the alarm, escalation and - after ticking *Change channels* - the escalation and notification channels; for sensors also condition and limit. Every field starts as *unchanged*: only what you fill in changes, everything else stays as it was for each monitor. If a value is invalid (e.g. less than 20 seconds), nothing changes. Condition and limit only go to sensors with a numeric value; state sensors (online, healthy ...) are skipped, the message says how many. Editors limited to locations can only select monitors of their locations.

Escalation

When an outage lasts too long, someone else should know as well, e.g. the management or a second on-call team. That is what **escalation** is for.

- **For all monitors:** *Settings → Notifications → Escalation*. There you choose after how many minutes still down (15 min to 8 h) it escalates, and to which channels. You add those channels beforehand as usual, e.g. an extra e-mail or a second Telegram chat.
- **Per monitor:** in the form under *Advanced → Escalation* you set another time, switch it off (*never escalate*) or choose own channels. Without an own choice the settings for all apply.
- The escalation arrives **once per outage**, with high priority: "🚨 Escalation: Firewall has been down for 30 min". When the outage is solved, the all-clear also goes to the escalation channels.
- Follow-up outages (see dependencies) and maintenance windows do not escalate. The history of the monitor shows *Escalated* with the channels that were reached.

History over months

On the page of a monitor you choose the range: **24 h, 3 d, 7 d** show every single check, **30 d, 90 d, 1 y** the summary.

- **Line:** the average, per 6 hours for 30 days, otherwise per day. Depending on the monitor this is the response time or the measured value, such as CPU or how full a drive is.
- **Area:** the range from the lowest to the highest value.
- **Strip below:** shows the availability. Green means always up or fine, yellow briefly down, red down for longer. Pointing the mouse at a section shows the date, percentage and values.

Outages in the chart: above the chart you see how often the monitor was down in the chosen range, for how long in total and the longest (for sensors: how often there was an alarm). Every outage has a red background, from the first failed check to the next good one. Below the chart the outages are listed with start, end, length and reason, for sensors with the highest (or lowest) value during the alarm. Pointing at a row highlights the outage in the chart; clicking into an outage in the chart takes you to its row.

Point instead of search: moving the mouse over the chart (on a phone: tap) shows a line at that spot and a box with time and value, the reason of a failed check, and within an outage its period. For sensors a dashed red line shows the alarm limit.

Single checks are kept for 7 days, the hourly summary for a good year. For the first months after the update to 0.9: older hours only know the average, so the area appears only for new data.

Dependencies: no alarm flood

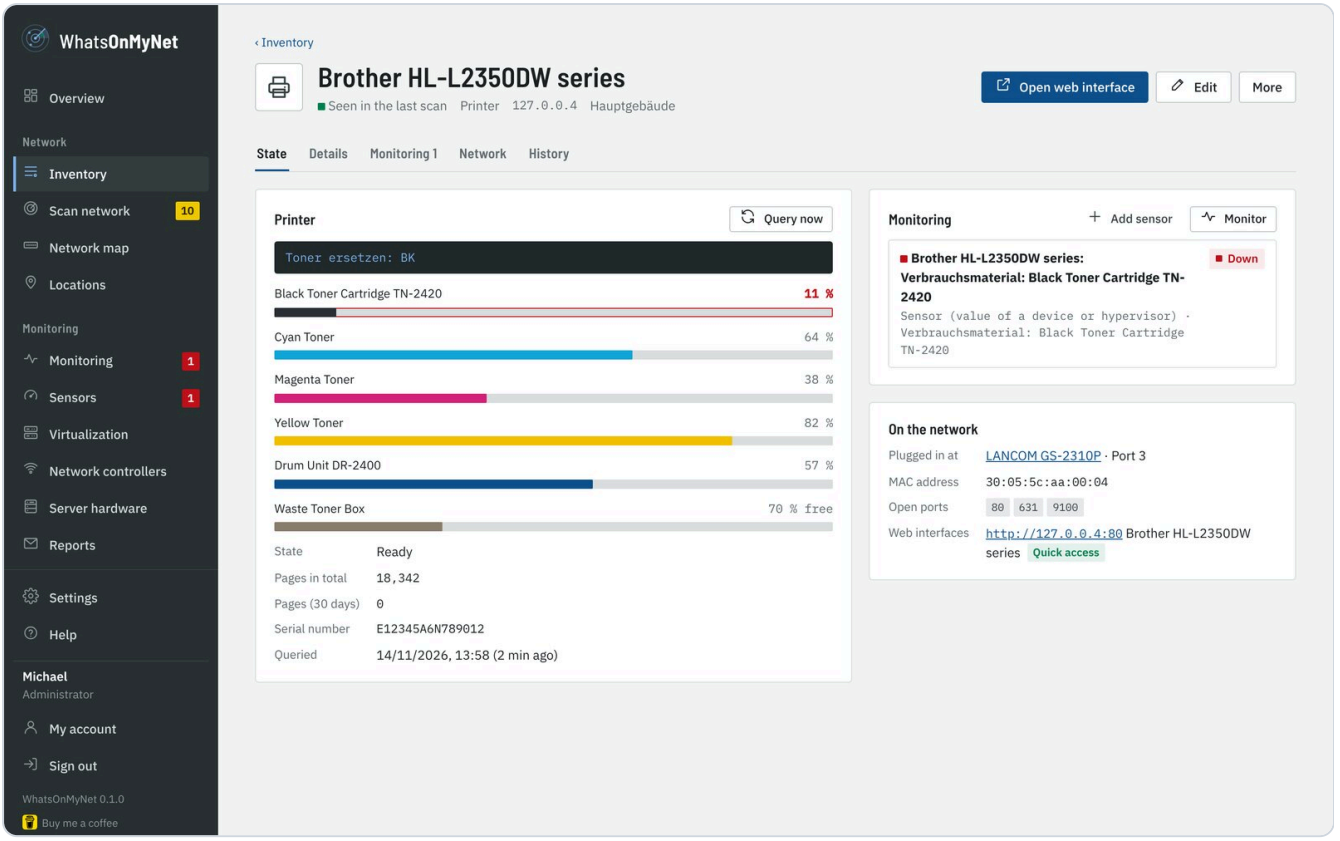
When a switch fails, every device behind it is unreachable too. So that twenty messages do not arrive at once, every monitor can **depend on** another one (form → *Depends on*, e.g. the ping of the switch or the internet router).

- While the monitor in front is down, the one behind it is a **follow-up outage** (hollow red dot) and sends nothing. Only the cause is reported.
- When both fail at the same time, the one behind waits briefly for the result of the one in front instead of raising its own alarm.
- When the monitor in front is back, the ones behind are checked again right away. Whatever still fails has a problem of its own and is reported as usual.
- Chains are possible (internet router → core switch → floor switch → printer). Circles are refused.

Automatically by switch port: When a switch is queried by SNMP, WhatsOnMyNet knows which switch a device is plugged in at. When you **Monitor** a device, the monitor of its switch is already chosen as dependency. The page of the switch monitor also tells how many monitored devices are plugged in at it and depend on nothing yet. **Put them behind this monitor** sets them all at once.

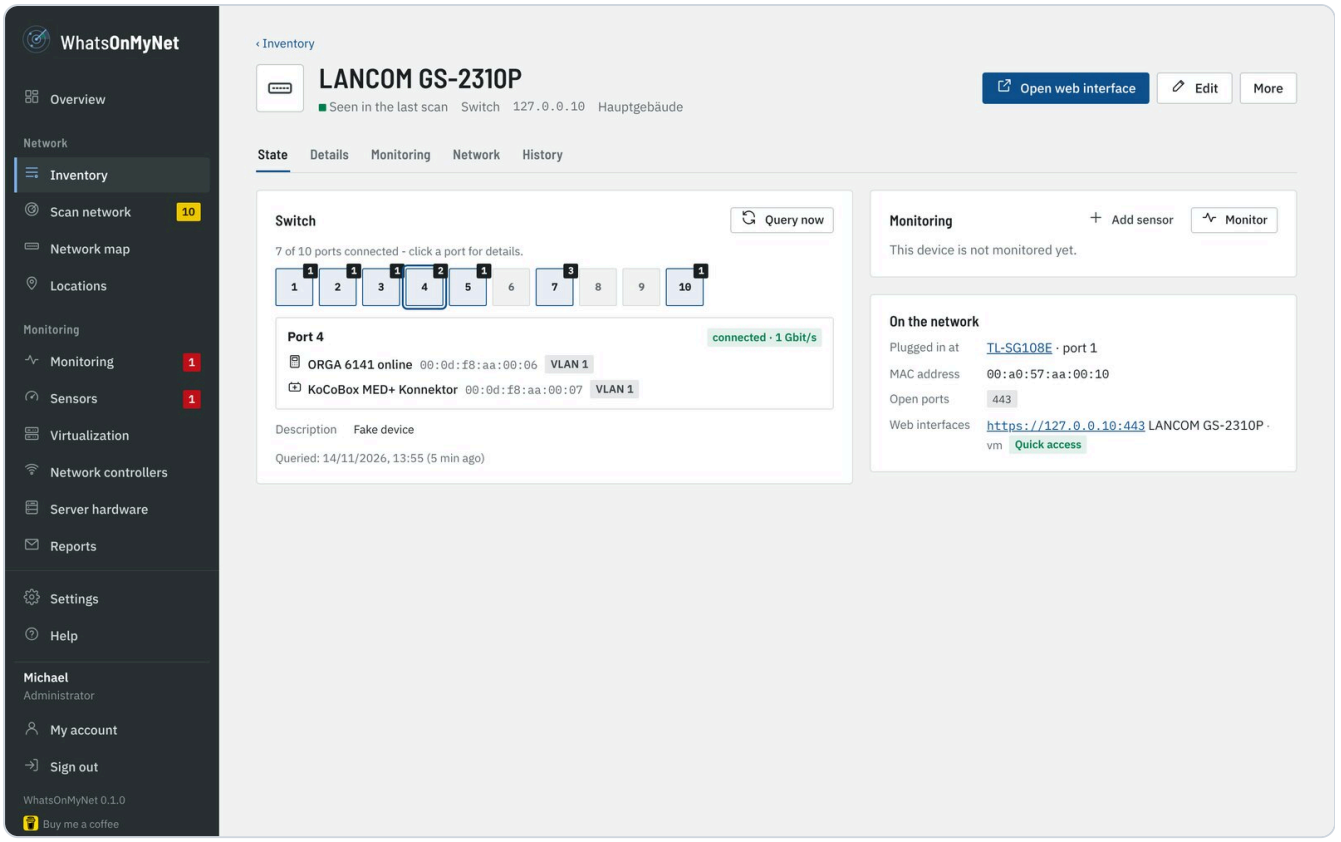
Printers, switches and NAS

Devices of type *Printer*, *Switch* and *NAS* are queried via SNMP (read-only). Set the community under *Settings* → *Scan*; SNMP v1/v2c has to be on at the device.

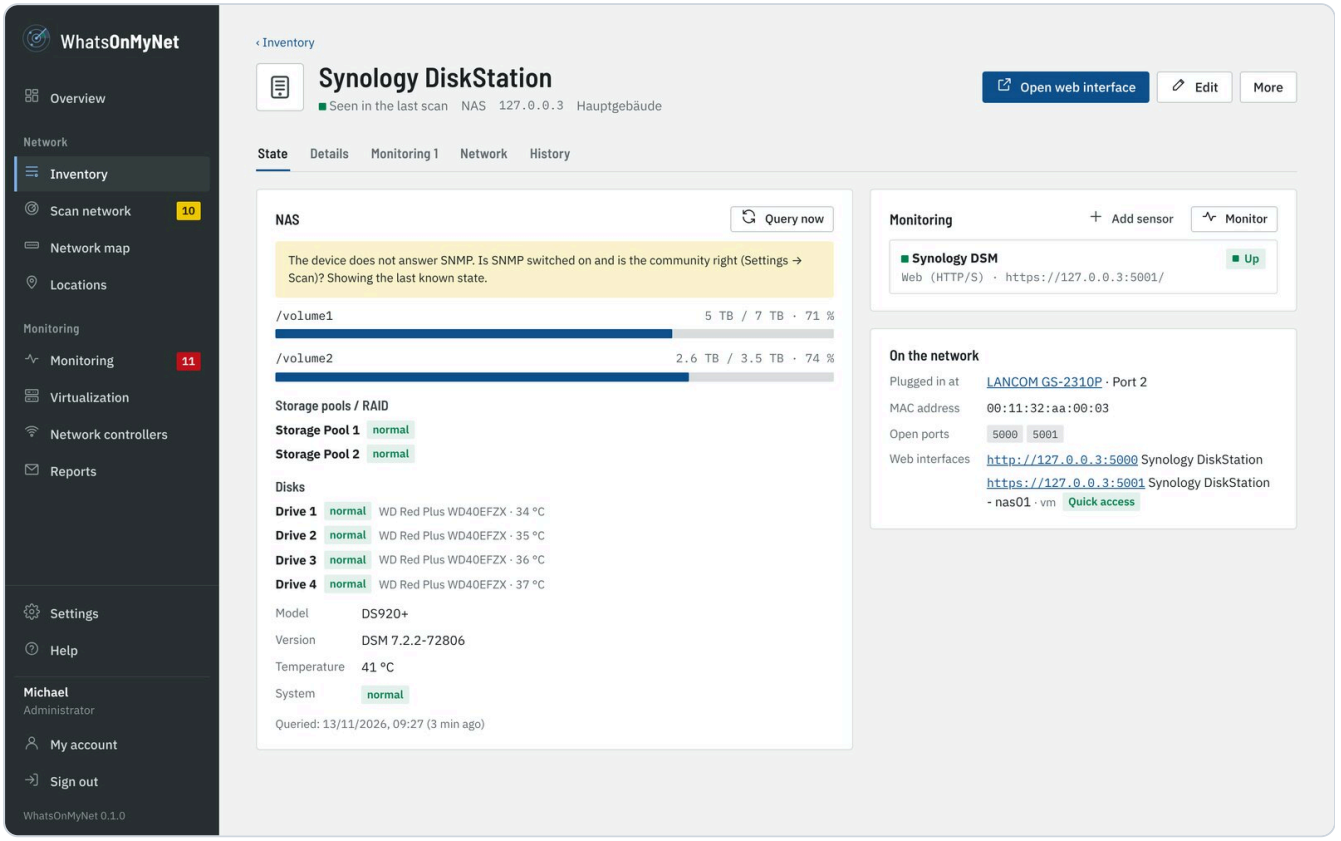


Printer

- **Printer**: toner or ink per colour, drum, waste toner, page counter, faults and display text. Below 15 % a notice is sent - once per cartridge.
- **Switch**: ports as tiles, which devices hang at which port (VLAN-aware), LLDP neighbours, firmware. Every device page shows "Plugged in at".
- **NAS**: volumes; on Synology also disks, RAID, temperature, fans, power supply and DSM update. Problems are reported - and again when everything is fine.
- **Taken into the inventory**: what the device says about itself - serial number, model, firmware or operating system (e.g. DSM version), for Synology/QNAP the manufacturer - goes right into the details of the device and stays current (e.g. after a firmware update). What you entered yourself with another value stays. The history shows every takeover.
- **Device exchanged**: if the device at the same IP reports another serial number, it is new hardware. Serial number, model, manufacturer and firmware then come from the new device, the old MAC address is dropped and replaced by the new one on the next scan (instead of a "new device"). The history notes the exchange - please check purchase date and warranty yourself.


WhatsOnMyNet
Overview
Network
Inventory
Scan network 10
Network map
Locations
Monitoring
Monitoring 1
Sensors 1
Virtualization
Network controllers
Server hardware
Reports
Settings
Help
Michael
Administrator
My account
Sign out
WhatsOnMyNet 0.1.0
Buy me a coffee
‹ Inventory
LANCOM GS-2310P
Seen in the last scan Switch 127.0.0.10 Hauptgebäude
Open web interface
Edit
More
State
Details
Monitoring
Network
History
Switch
Query now
7 of 10 ports connected - click a port for details.
1 2 3 4 5 6 7 8 9 10
Port 4
connected · 1 Gbit/s
ORGA 6141 online 00:0d:f8:aa:00:06 VLAN 1
KoCoBox MED+ Konnektor 00:0d:f8:aa:00:07 VLAN 1
Description Fake device
Queried: 14/11/2026, 13:55 (5 min ago)
Monitoring
Add sensor
Monitor
This device is not monitored yet.
On the network
Plugged in at TL-SG108E · port 1
MAC address 00:a0:57:aa:00:10
Open ports 443
Web interfaces https://127.0.0.10:443 LANCOM GS-2310P · vm Quick access

Switch


WhatsOnMyNet
Overview
Network
Inventory
Scan network 10
Network map
Locations
Monitoring
Monitoring 11
Virtualization
Network controllers
Reports
Settings
Help
Michael
Administrator
My account
Sign out
WhatsOnMyNet 0.1.0
‹ Inventory
Synology DiskStation
Seen in the last scan NAS 127.0.0.3 Hauptgebäude
Open web interface
Edit
More
State
Details
Monitoring 1
Network
History
NAS
Query now
The device does not answer SNMP. Is SNMP switched on and is the community right (Settings → Scan)? Showing the last known state.
/volume1 5 TB / 7 TB · 71 %
/volume2 2.6 TB / 3.5 TB · 74 %
Storage pools / RAID
Storage Pool 1 normal
Storage Pool 2 normal
Disks
Drive 1 normal WD Red Plus WD40EFZX · 34 °C
Drive 2 normal WD Red Plus WD40EFZX · 35 °C
Drive 3 normal WD Red Plus WD40EFZX · 36 °C
Drive 4 normal WD Red Plus WD40EFZX · 37 °C
Model DS920+
Version DSM 7.2.2-72806
Temperature 41 °C
System normal
Queried: 13/11/2026, 09:27 (3 min ago)
Monitoring
Add sensor
Monitor
Synology DSM
Up
Web (HTTP/S) · https://127.0.0.3:5001/
On the network
Plugged in at LANCOM GS-2310P · Port 2
MAC address 00:11:32:aa:00:03
Open ports 5000 5001
Web interfaces http://127.0.0.3:5000 Synology DiskStation
https://127.0.0.3:5001 Synology DiskStation - nas01 · vm Quick access

NAS

UPS (uninterruptible power supply)

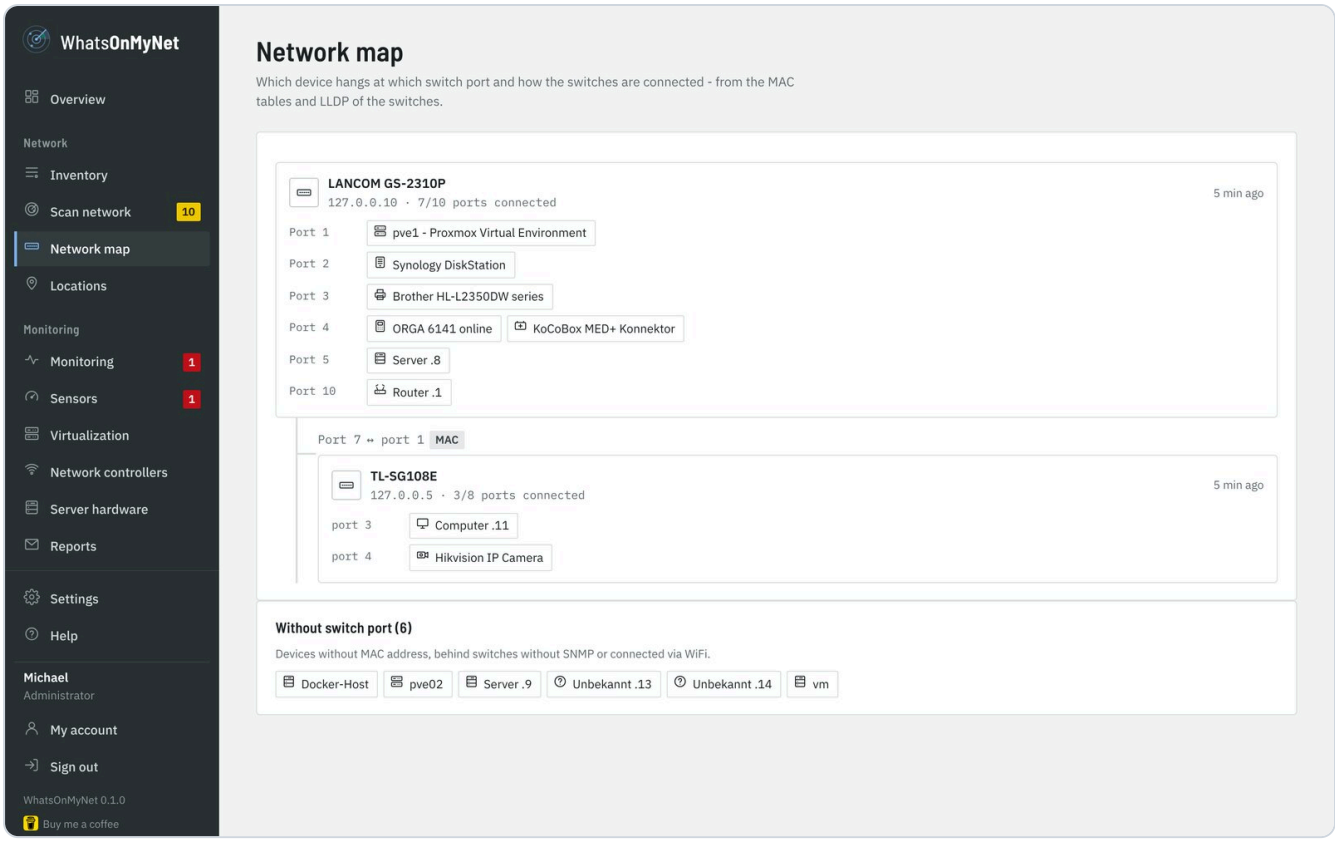
WhatsOnMyNet reads UPS values along the way, without anything to set up. The values are always the same: **on mains or battery**, **battery charge**, **how long the battery lasts** and **load**. With **Add sensor** you pick e.g. "UPS: mains or battery" (alarm as soon as it runs on battery) or "UPS runtime left (min)".

- **UPS at a Synology** (USB or network): when the UPS service of the Synology is on, the NAS area shows a *UPS* section. It is read over the same SNMP as the NAS. You do not have to add WhatsOnMyNet as a trusted device in the UPS server.
- **UPS with its own network card:** APC, or any card with the standard UPS-MIB, e.g. Eaton, CyberPower or Riello. Set the device in the inventory to type *UPS*, then it is read by SNMP like switches and NAS. Serial number, model and firmware go into the inventory.
- **UPS at a Linux machine:** when NUT (`upsc`) or apcupsd (`apcaccess`) already runs there, the machine script reads the values locally.
- **UPS on USB at a Windows machine:** Windows sees it like a battery, and the machine script sends the values along. On laptops this is the built-in battery, so the section is called *UPS or battery* there.

A UPS at a **QNAP** does not give its values over SNMP, so WhatsOnMyNet shows no UPS there. Shutting down and control stay with the UPS software; WhatsOnMyNet only reads.

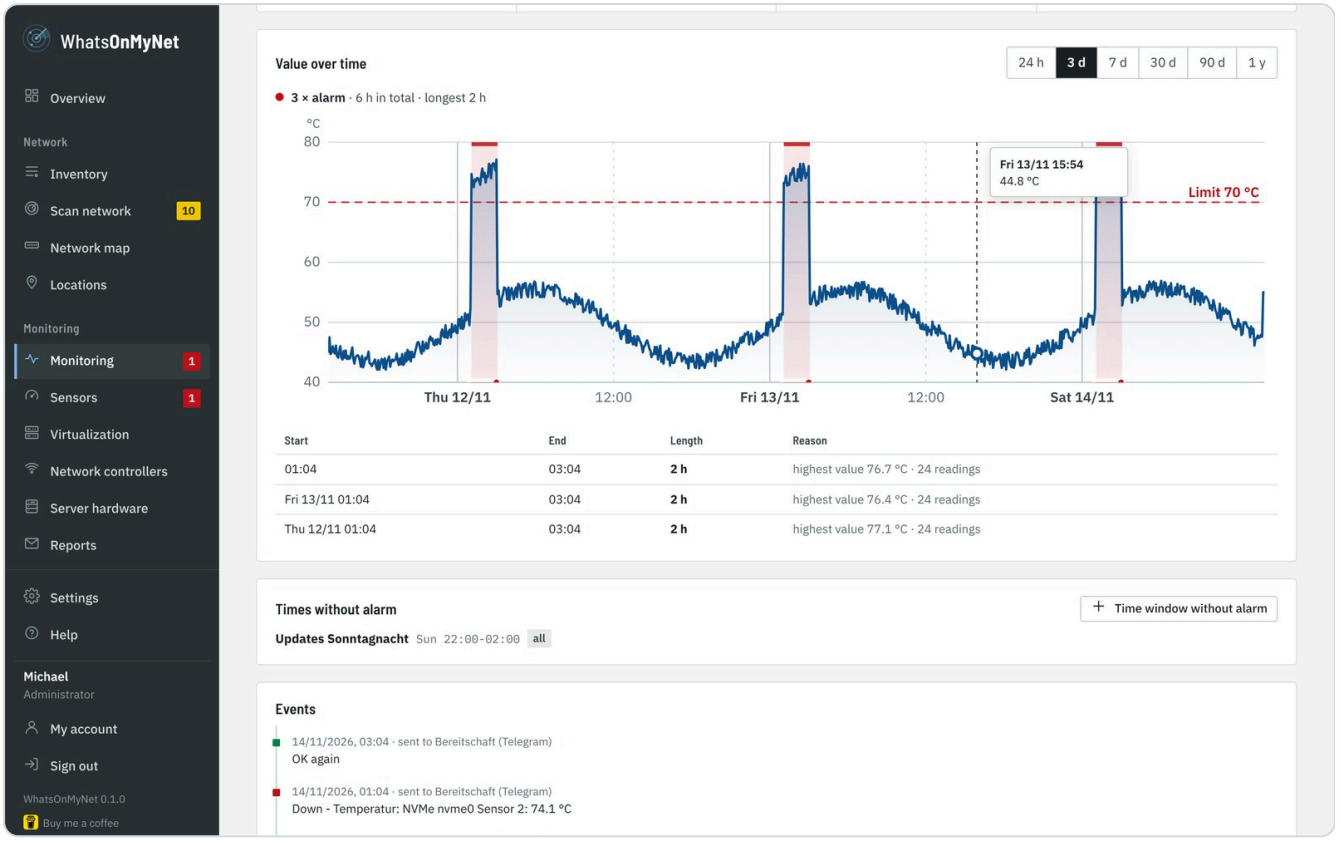
Network map

The *Network map* shows from MAC tables and LLDP which device hangs at which port and how the switches are connected. Simple switches without SNMP show up as a branch behind the port they are plugged into. Behind routers the scan reads the ARP table of the routers via SNMP, so the MAC address stays the identity there too.



Network map

Sensors



Sensor with limit

Every value a device delivers can become a monitor of its own - with history, alarm and maintenance windows. All sensors are listed under *Monitoring → Sensors*. "Add sensor" is on the device page, at the host under *Virtualization* and under *Monitoring → Sensors*. It is always the same dialog: choose device or host, then value and condition. Values another sensor already watches are not offered again (also not through the hypervisor when the Linux script delivers the same value). Examples:

- "toner cyan less than or equal 10 %"
- "usage /volume1 greater than or equal 85 %"
- "temperature drive 2 greater than 50 °C"
- "port 10 (uplink) connected"

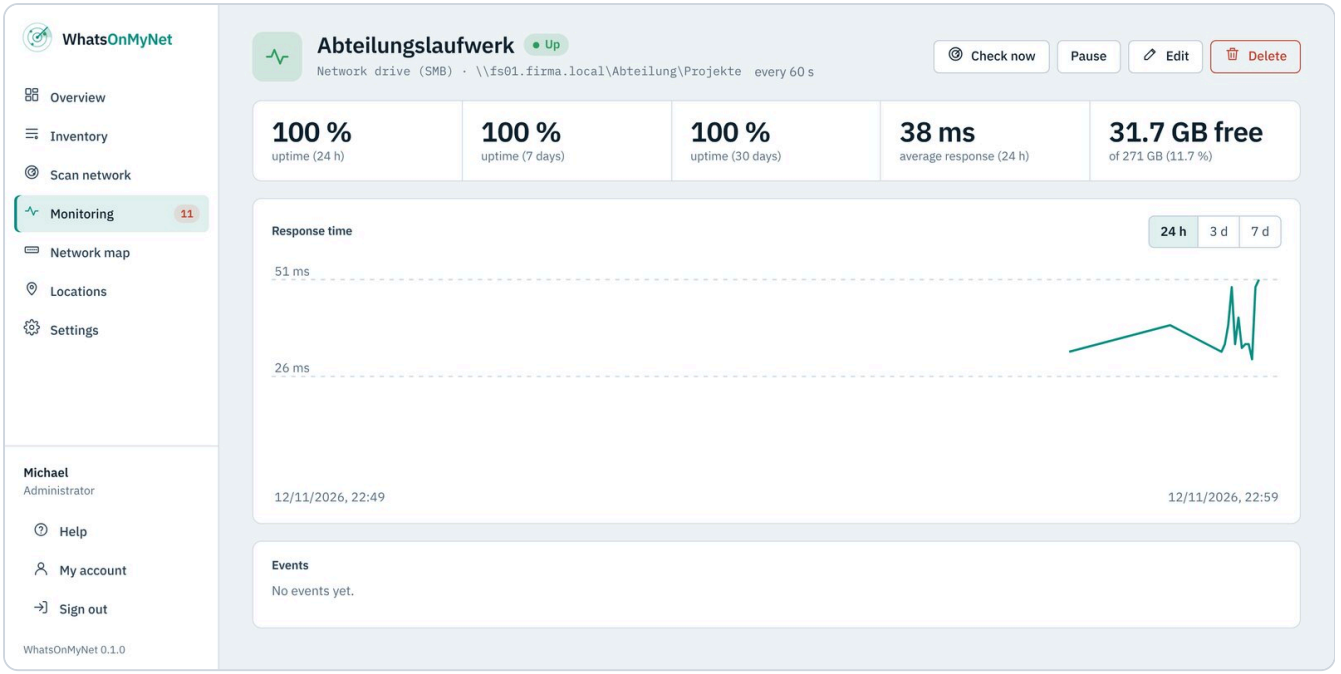
Proxmox nodes: a node that is also in the inventory (same name, e.g. `pve2` or `pve2.firma.local`) has one common list - whether opened from the device or from the host under Virtualization: "Node pve2 online" from Proxmox and, if the Linux script runs there, every temperature, CPU, RAM, drives and services from the script. What both deliver (CPU, RAM, the highest temperature) is listed only once. Under Virtualization the list is grouped by node. Likewise a server with a management chip (Redfish) shows its hardware values and a Proxmox Backup Server its datastores and backups at the device.

When editing, an existing sensor can be switched to any other value of this list, for example from "Temperature pve2" to "Temperature: NVMe nvme0 Sensor 2". The name follows ("pve2: Temperature: NVMe nvme0 Sensor 2") - unless it is changed by hand in the same dialog.

The kind **SNMP value (OID)** reads any SNMP value (e.g. runtime left of a UPS) and compares with less, greater, equal or "contains text".

In the monitor list the column **Now** shows the value right away: number and unit, a small bar for percentages, "used of total" for storage and RAM, and the alarm limit ("alarm from 90 %"). Close to the limit the value turns yellow, at the alarm red. Nodes and VMs show their state with CPU and RAM, network drives the free space.

Network drives (SMB)



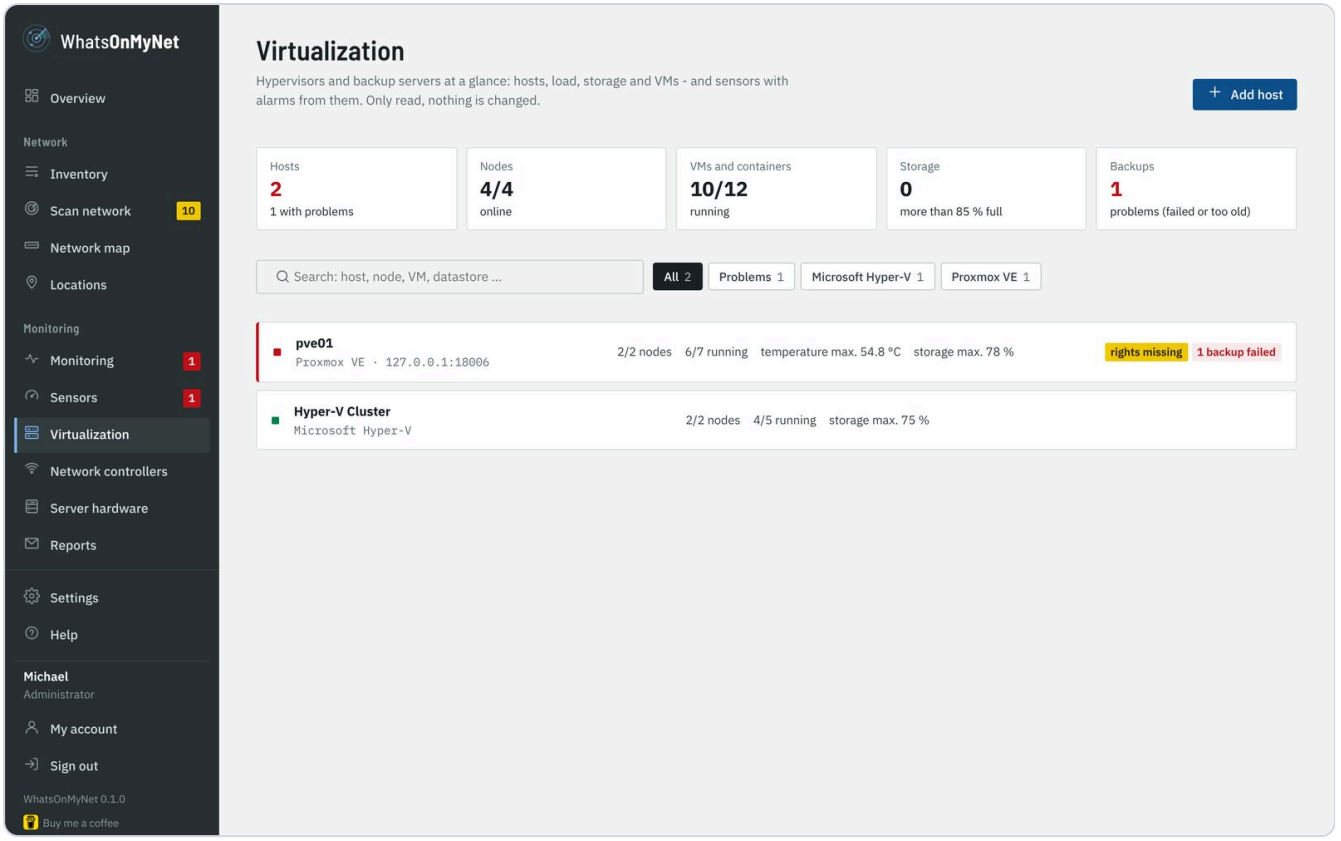
Network drive

The kind **Network drive (SMB)** really signs in to a share - not just to the open port. So you also notice when the password expired, the rights changed or a folder is missing.

1. Create an account of its own with **read rights** on the share (in AD e.g. `COMPANY\monitor`, on a NAS a local user).
2. *Add monitor* → kind **Network drive (SMB)**. Just paste the path, e.g. `\\fs01\Department\Projects` - server, share and folder are taken from it.
3. Enter user, domain (empty for NAS accounts) and password. Optional: warn below … % or … GB free.

The check runs over SMB 2/3: sign in, read the share (or the folder or file), ask for the free space. WhatsOnMyNet writes nothing to the share. The detail page shows the free space; reports have the section *Network drives* for it.

Proxmox VE and Backup Server



Proxmox

WhatsOnMyNet reads Proxmox VE and the Proxmox Backup Server through their built-in interface (API). Nothing is installed on Proxmox. WhatsOnMyNet gets a user of its own there that may **only read**, and a key for it (API token). The dialog *Monitoring → Virtualization → Add host → Proxmox VE* shows the same steps, with a button to copy.

Cluster: entering one node is enough. WhatsOnMyNet asks the cluster which nodes there are, at which address they are reachable and which certificate they have (again every 6 hours). If the entered node does not answer, the next node is asked - with the same token (it is valid in the whole cluster) and only after its certificate has been checked. The virtualisation page then shows "read through node pve2". The Proxmox Backup Server knows no cluster: each one is entered on its own.

Temperatures of the nodes: the Proxmox interface knows no temperatures. If a node also runs the Linux script (see *Windows and Linux machines by script*), its highest temperature is shown at the node on the virtualisation page. A sensor for it: at the Proxmox host or at the device **Add sensor** → *Temperature: the highest of all sensors* (alarm from 85 °C, can be changed; single sensors with the chip's warning limit); single sensors (CPU, NVMe ...) are in the same list under "Node pve1". They are matched by name: node `pve1` ↔ machine `pve1` or `pve1.firma.local`.

Step 1: prepare Proxmox (once)

With commands (recommended) - Proxmox VE

1. In Proxmox click a node on the left and open **>_ Shell** at the top. One node is enough - in a cluster it applies to all.
2. Copy these three lines, paste them into the shell and press Enter:

```
pveum user add whatsonmynet@pve --comment "WhatsOnMyNet (read only)"
pveum acl modify / --users whatsonmynet@pve --roles PVEAuditor
pveum user token add whatsonmynet@pve womn --privsep 0
```

3. The output shows a table. The row `value` holds the **secret** (a long string with dashes). Copy and keep it - Proxmox shows it only this once.

With commands - Proxmox Backup Server

1. In the Backup Server open *Administration* → *Shell* on the left.
2. Paste these four lines and press Enter:

```
proxmox-backup-manager user create whatsonmynet@pbs --comment "WhatsOnMyNet (read only)"
proxmox-backup-manager acl update / Audit --auth-id whatsonmynet@pbs
proxmox-backup-manager user generate-token whatsonmynet@pbs womn
proxmox-backup-manager acl update / Audit --auth-id 'whatsonmynet@pbs!womn'
```

3. After the third command `"value"` holds the **secret**. Copy it - it is shown only once.

By clicking - Proxmox VE

1. *Datacenter* → *Permissions* → *Users* → *Add*: user name `whatsonmynet`, realm *Proxmox VE authentication server*, any password → *Add*.
2. *Datacenter* → *Permissions* → *Add* → *User Permission*: path `/`, user `whatsonmynet@pve`, role `PVEAuditor` → *Add*.
3. *Datacenter* → *Permissions* → *API Tokens* → *Add*: user `whatsonmynet@pve`, token ID **only** `womn` (no @ or !), **untick** *Privilege Separation* → *Add*.
4. Proxmox now shows the *Secret* - only this once. Copy and keep it.

On the Backup Server it works the same under *Configuration* → *Access Control*: user `whatsonmynet` (realm *Proxmox Backup authentication server*), tab *API Token* with token name `womn`, then under *Permissions* the role `Audit` on the path `/` twice - once for the user `whatsonmynet@pbs`, once for the token `whatsonmynet@pbs!womn`.

Step 2: enter it in WhatsOnMyNet

1. *Monitoring* → *Virtualization* → **Add host** → *Proxmox VE* or *Proxmox Backup Server*.
2. Give it a name.
3. Enter the **address** the way you open Proxmox in the browser, e.g. `https://192.168.1.10:8006` (Backup Server: port `8007`).
4. Click **Fetch certificate**. WhatsOnMyNet shows the fingerprint of the Proxmox certificate. Compare it - Proxmox VE: click the node → *System* → *Certificates* → row `pve-ssl.pem`;

Backup Server: *Dashboard* → *Show Fingerprint*. If it matches, **Fingerprint matches - trust**.

5. The **token ID** is already filled in (`whatsonmynet@pve!womn` or `whatsonmynet@pbs!womn`). Paste the copied secret into **Token secret**.
6. **Save and connect.** After a few seconds nodes, storage, VMs and containers and the backups appear.

Adding sensors

In the detail view **Add sensor** - e.g. "VM 101 running", "storage local-lvm below 85 %" or "last backup older than 26 hours". Every sensor is a monitor with history and notification.

If something does not work

- **"tokenid: value does not match the regex pattern" in Proxmox**: when adding the token enter only `womn` - without user, @ and !.
- **"sign-in refused (401)"**: token ID or secret are wrong. Delete the token in Proxmox, add it again and enter the new secret.
- **Notice "The token lacks read rights"**: usually *Privilege Separation* stayed ticked on the token. The notice shows two commands - paste them into the Proxmox shell, Enter, then **Query now**.
- **"certificate changed"**: Proxmox got a new certificate. *Edit* → **Fetch certificate** → confirm.

Many hosts at a glance

From two hosts on, the page *Virtualization* (as well as *Network controllers* and *Server hardware*) shows the situation in numbers at the top: hosts with problems, nodes online, running VMs, full storage, backup problems. Below there is **one line per host** with the key values and what is wrong. Hosts with problems come first. A click on the line unfolds the details (the browser remembers that until the tab is closed).

The **search** finds hosts, nodes, VMs (name or ID), datastores and backup groups. The buttons filter by *Problems* or by kind (Proxmox VE, Backup Server ...).

More hypervisors: VMware, XCP-ng, Hyper-V, Nutanix, oVirt, KVM

Under *Monitoring* → *Virtualization* → *Add host* further hypervisors are offered beside Proxmox. All appear the same way: hosts with CPU and RAM, storage, VMs - and sensors can be added from everything ("host online", "CPU esx01", "storage vsan", "VM dc01 running"). It only reads.

Asked through their interface - an account that may only read is enough; enter the address, **Fetch certificate**, compare the fingerprint, enter the account:

Hypervisor	Account with read rights	Address
VMware ESXi / vCenter	user with role *Read-only*, given at the top of the inventory, "propagate to children"	vCenter or ESXi, e.g. `https://vcenter.firma.local`

Hypervisor	Account with read rights	Address
XCP-ng / XenServer	with AD: role *read-only*; without AD there is only `root`	pool master, e.g. `https://xcp01.firma.local`
Nutanix AHV	Prism user with role *Viewer*	Prism with port, e.g. `https://prism.firma.local:9440`
oVirt / RHV / OLVM	`whatsonmynet@internal` with role *ReadOnlyAdmin* (commands are in the dialog)	engine, e.g. `https://engine.firma.local`

Hyper-V and KVM/libvirt report on their own. Neither has an interface to ask. Instead a small script on the host sends load, volumes or storage pools and VMs every 5 minutes. It only reads and needs no open port on the host.

1. *Add host* → **Microsoft Hyper-V** or **KVM / libvirt**, give a name, save.
2. The dialog shows the script with a key of its own (only this once) - **Download script**.
3. Run it once, it sets itself up. **Hyper-V** as administrator: `powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\womn-hyperv.ps1` (scheduled task *WhatsOnMyNet-HyperV*, every 5 minutes as SYSTEM). In a cluster on every node - all nodes land in the same entry. **KVM:** `sh womn-libvirt.sh` as root or as a user of the group `libvirt` (cron entry, every 5 minutes) - otherwise `virsh` may not see the VMs of the system. It needs `virsh`, `awk` and `curl`; it reads with `virsh -r`. Removing again: the **uninstall script** from the dialog or *Settings* → *Machine scripts* - it also removes the setup of version 0.6.0.
4. After a few seconds the host appears. If it does not report for more than 15 minutes it counts as offline - a sensor "host online" then raises the alarm.

If the key is lost or should be renewed: **New script** - the old one stops working then.

With the self-signed certificate of WhatsOnMyNet the script trusts exactly its key (built in); with a certificate of a real CA it checks as usual. When WhatsOnMyNet renews its certificate (30 days before it expires, after a new IP address or `WOMN_BASE_URL`), the key stays the same and the scripts go on reporting. Exception: Windows scripts made with versions up to 0.10.1 only know the certificate itself - create them once more. Scan agents with `WOMN_AGENT_CA` need the new value from *Settings* → *Scan agents* after a renewal.

Network controllers: UniFi, Omada, MikroTik

Under *Monitoring* → *Network controllers* → *Add controller* WhatsOnMyNet reads the widespread network systems - with an account that may **only read**; enter the address, **Fetch certificate**, compare the fingerprint, enter the account. The dialog shows the steps for each system.

System	Account	Address
UniFi Network - UniFi OS consoles (UDM, UCG, Cloud Key Gen2+) or the installed Network app	local account (without two-factor) with role *View only*/*Site Viewer*	console, or `https://server:8443`
TP-Link Omada - software controller, OC200/OC300	account with role *Viewer*	usually `https://server:8043`

System	Account	Address
MikroTik RouterOS 7 - every router/switch on its own	user in a group with `read,api,rest-api`	router with service *www-ssl*

The page shows all access points, switches and gateways of each controller with model, IP, firmware (with "Update" when a new one is ready), clients and state.

Matched with the inventory: every device that is in the inventory by its MAC address gets model, serial number, firmware, manufacturer and the right **type** from the controller - by the same rules as for SNMP (empty or still device-reported details are filled or kept current, your own different entries stay, an exchange is recognised). Devices that are not in the inventory yet carry the note *not in the inventory* - adopt them by scan.

Sensors: "AP hall online", "Clients on AP hall", "CPU ...", "RAM ...", "Clients in total" and "Devices with firmware update".

The **scan** now also recognises these devices better by their model names: UniFi (USW = switch, UAP/U6/U7 = access point, UDM/UCG/USG = gateway, Network app = server), Omada (EAP = access point, ER = gateway, OC200/OC300 = controller), MikroTik (CRS = switch, cAP/wAP = access point) and Aruba Instant On.

Server hardware: iDRAC, iLO, XClarity, Supermicro (Redfish)

Fans, power supplies, temperatures, RAID controllers and the drives behind them are usually unknown to the operating system. The management chip of the server knows them, even while the server is switched off. All current chips speak the standard **Redfish**: Dell iDRAC, HPE iLO, Lenovo XClarity Controller, Supermicro, Fujitsu iRMC, Cisco CIMC.

1. Add an account of its own in the management chip that may **only read**: iDRAC role *Read Only*, iLO only the right *Login*, XClarity role *ReadOnly*, Supermicro role *User*.
2. *Monitoring* → *Server hardware* → **Connect management chip**: the address of the chip (not of the server), **Fetch certificate** and confirm the fingerprint, user and password.

Shown are:

- overall health and power state, consumption
- model, serial number (at Dell the service tag), BIOS, processors, memory, firmware of the chip
- fans, temperatures with warning limit, power supplies
- RAID controllers, volumes and drives, with life left and predicted failure

Whatever is not fine is listed at the top. WhatsOnMyNet matches the server to **its device in the inventory** by the MAC addresses of its network cards or by the serial number. Its page then shows the *Server hardware* area, and manufacturer, model and serial number are taken over.

Sensors (*Add sensor*): overall health, server switched on, every fan, temperature, power supply, drive (also life left), volume, and power consumption. It is asked every 5 minutes like Proxmox, read only.

Windows and Linux machines by script (without agent)

What is the script for? Many values of a machine are known only to its operating system: temperatures of CPU and NVMe, the health of the disks (SMART), services, pending updates. Big servers deliver part of it through their management chip (iDRAC, iLO, XClarity - see *Server hardware*). A mini PC, a self-built server or a Proxmox node in a homelab has nothing like that - here the script fetches this information. For Proxmox: the API delivers nodes, VMs, storage and backups; temperatures, disks, services and updates of the nodes only come with the script.

The tip in the program: machines, servers, VMs and hypervisors with Windows or Linux that have no system values yet show the tip "Monitor with script" on the device page; under *Virtualisation* it appears for Proxmox nodes without the script. It disappears by itself as soon as the device delivers values - through the script or through the server hardware (iDRAC, iLO ...). If you deliberately do not want the script for a device, **Do not show again** hides the tip there for good; *More → Monitor with script* still works at any time.

Servers and PCs can be monitored without an agent: a small script on the machine reports **CPU, RAM, drives, uptime and services** every 5 minutes, plus a connected **UPS** (see above).

On top comes what an administrator wants to know about a server. Everything is on the device page in the *System* area, and every value can become a sensor with alarm:

- **Updates and protection:** pending updates (how many important/security), whether a restart is pending, virus protection (Defender or another product registered in the Security Center: active, signatures current) and firewall (Windows: domain/private/public, Linux: ufw or firewalld). The update search runs only every 6 hours and after the report, so that the 5-minute report stays quick. It only searches and never installs anything.
- **Disks:** model, serial number, health, temperature, wear and power-on hours. On Linux this needs `smartctl` and root, otherwise there is only model, serial number and size.
- **Events:** critical entries and errors of the last 24 hours (Windows: system log, Linux: journal), the newest ten with text.
- **Certificates** of the machine (Windows, store "Personal" of the computer), with days until they expire.
- **Hardware:** manufacturer, model, serial number, BIOS, processors, memory modules and network cards. Manufacturer, model and serial number also go into the inventory. On Linux, serial number and memory modules only come as root (`dmidecode`).
- **Temperatures (Linux):** CPU, NVMe, chipset and - depending on the mainboard - further sensors, straight from the kernel (`/sys/class/hwmon` , the same source as `sensors`). Nothing has to be installed, not even lm-sensors, and root is not needed. The warning limit is the one the chip names itself, otherwise 80 °C. Sensors: *Temperature* (one sensor) or *Temperature: the highest* (all together, alarm from 85 °C). If a mainboard sensor is missing, the kernel does not load its driver by itself - `sensors-detect` from lm-sensors finds it and adds it to `/etc/modules` .

Fans, power supplies, further temperatures and RAID controllers are usually unknown to the operating system. They come from the management chip of the server (see *Server hardware*

above).

The script installs nothing, loads nothing, opens no port and needs no account - it only sends to WhatsOnMyNet. (On Windows the update search asks the WSUS server or Windows Update, like Windows itself.) On Windows it is a PowerShell script (runs as SYSTEM), on Linux a POSIX sh script (needs `sh`, `awk` and `curl`, root is not needed).

A single machine: *Monitoring* → **Add monitor** → **Machine by script (Windows/Linux)** → choose the machine. On the device page the same works with **Monitor** (kind of check *Machine by script*) or *More* → **Monitor with script**. Then Windows or Linux → **Create script**.

1. **Download script** (it holds the address and the key - shown only this once).
2. Run it once - it sets itself up: - **Windows** as administrator: `powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\womn-host.ps1` - it puts itself into `C:\ProgramData\WhatsOnMyNet` (readable only for SYSTEM and administrators) and adds the scheduled task *WhatsOnMyNet-Host*. - **Linux**: `sh womn-host.sh` - as root into `/usr/local/sbin` with `/etc/cron.d/whatsonmynet-host`, otherwise into `~/.local/bin` with an entry in the own crontab.
3. After a few seconds the device page shows the section **System**. **Add sensor** turns it into monitors with alarms: "Machine reports", "CPU load", "Memory", "Drive C:", "Failed services" or "Service ... running".

Many machines at once: *Settings* → *Machine scripts* → **Shared script**. You roll out this one script by software distribution, a startup script by group policy or Ansible. Each report is matched to its device in the inventory (MAC address, machine name, IP). Machines that are not in the inventory yet show up there under *Unknown machines* - nothing is created on its own.

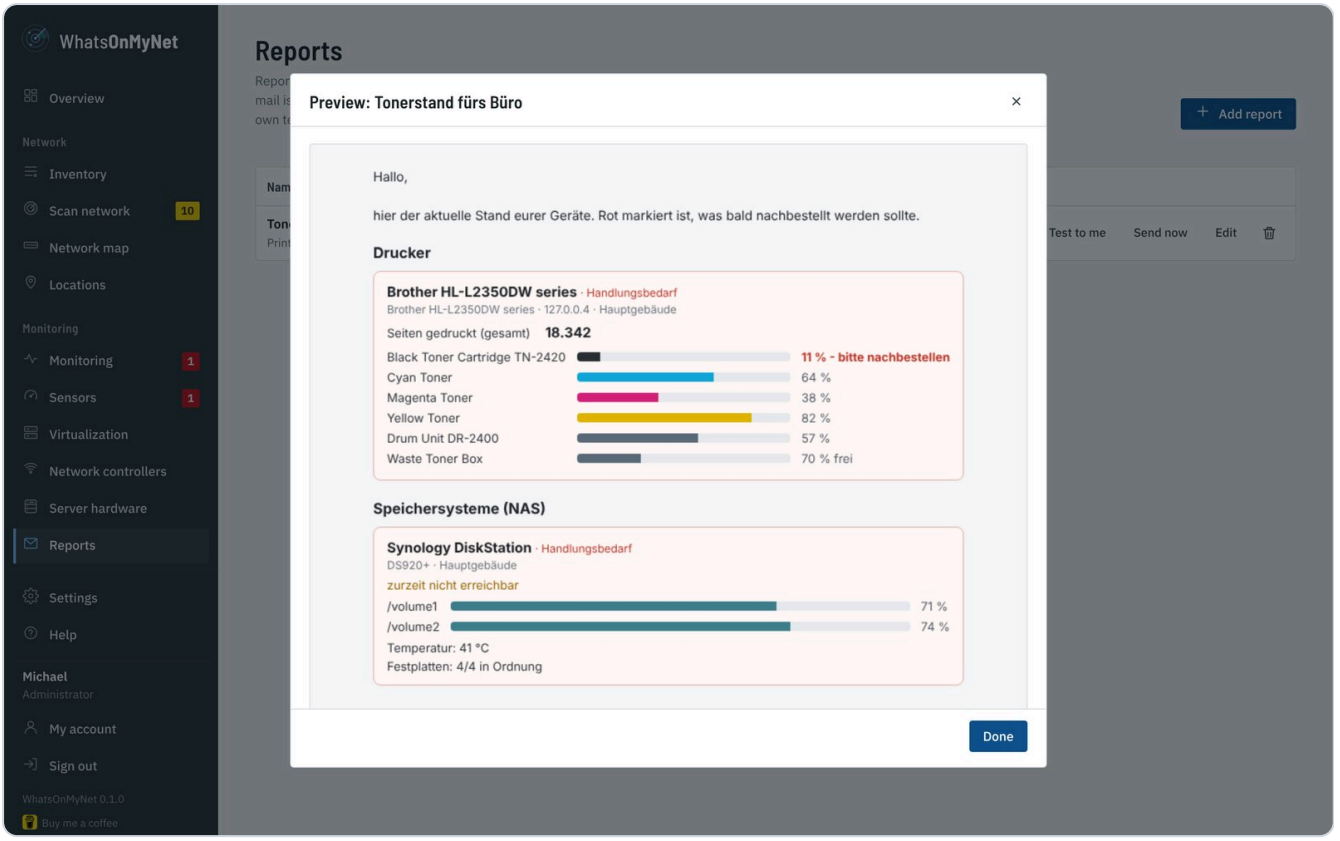
Failed services: Windows - services that start automatically and stopped with an error code (trigger-start services that stopped normally do not count); Linux - systemd services in the state *failed*.

New version of the script: when an update brings new values (e.g. the UPS), create the current script under *Settings* → *Machine scripts* in the row of the existing entry with **New script** and simply run or roll it out again. It overwrites the old copy and the scheduled task. The old key is blocked from then on, so roll it out right away.

Removing again: every script comes with an **uninstall script** (in the script dialog under *Remove again* and under *Settings* → *Machine scripts*). It removes the scheduled task or cron entry and the script - run or distribute it the same way. Afterwards the machine is as before. Deleting or recreating a script under *Settings* → *Machine scripts* locks the old key.

If a machine does not report for 15 minutes it counts as offline - the sensor "Machine reports" then raises the alarm.

Reports



Report

Under *Monitoring → Reports* reports go out by e-mail at set times - e.g. the toner levels to the office every Monday at 8:00.

- Content: printers, NAS, switches, network drives; can be limited to locations; optionally only devices that need action.
- **Devices**: all devices of the chosen content - or only chosen ones, e.g. a report of its own for the printer in the front office.
- Printers show the **pages printed** (in total and in the report period, e.g. "in the last 7 days") and - if the printer reports it - how many pages the toner still lasts.
- The mail is **neutral**: no link, no program name - only your sender name, subject (`{week}` , `{date}`), introduction, the devices and your closing.
- "Preview", "Test to me" and "Send now" right in the list.

Users and rights

- **Roles**: *Administrator* (everything), *Editor* (scan, adopt, maintain, monitors), *Read only*. Editors only scan private networks (10.x, 172.16-31.x, 192.168.x, 100.64-127.x) and the subnets of the locations; public addresses only administrators.
- **Locations**: editors and viewers can be limited to locations - they then only see and change devices, scans and monitors of these.

- **Active Directory**: sign-in with the Windows account, see the next section.
- **Two-factor sign-in** (*My account*): scan the QR code with an authenticator app, confirm a code, keep the recovery codes. It can be made mandatory for administrators under *Settings* → *General*.

Sign-in with Active Directory

Staff sign in with their Windows account. AD groups decide which role they get. Local accounts keep working; for the same name the local account wins, so the local admin stays the emergency access.

Step 1: prepare AD (once)

1. Create a **service account**, e.g. `svc-womn`. It needs no special rights, a normal domain account with a password is enough.
2. Create up to three **groups**, e.g. `WOMN-Admins`, `WOMN-Editors` and `WOMN-Read`, and put the staff into them. Groups in groups count.

Step 2: enter it in WhatsOnMyNet

1. *Settings* → *Active Directory*.
2. **Server**: name of the domain controller as it is in the certificate, e.g. `dc01.company.local`. Encryption **LDAPS** (port 636) or **StartTLS** (port 389). Without encryption it does not work.
3. **Certificate**: leave trust on *Fetched certificate* and click **Fetch certificate from DC**. Compare the fingerprint with the certificate on the DC (`certlm.msc` → *Personal* → open the certificate → *Details* → *Thumbprint*) and confirm. If the root CA is missing, WhatsOnMyNet gets it itself (via the AIA address in the certificate or, after confirming, from AD).
4. **Directory**: leave the base DN empty, the test finds it. Enter the service account, e.g. `svc-womn@company.local`, and its password.
5. **Groups**: simply enter the group names, e.g. `WOMN-Admins`. Members of several groups get the highest role; members of none cannot sign in.
6. **Test connection**, best with your own user name. Every step gets ✓ or × with the reason: connection, service account, base DN, groups, user, role and, if you like, the password.
7. When everything is ✓: tick *Allow sign-in through Active Directory* and **Save**.

If something does not work

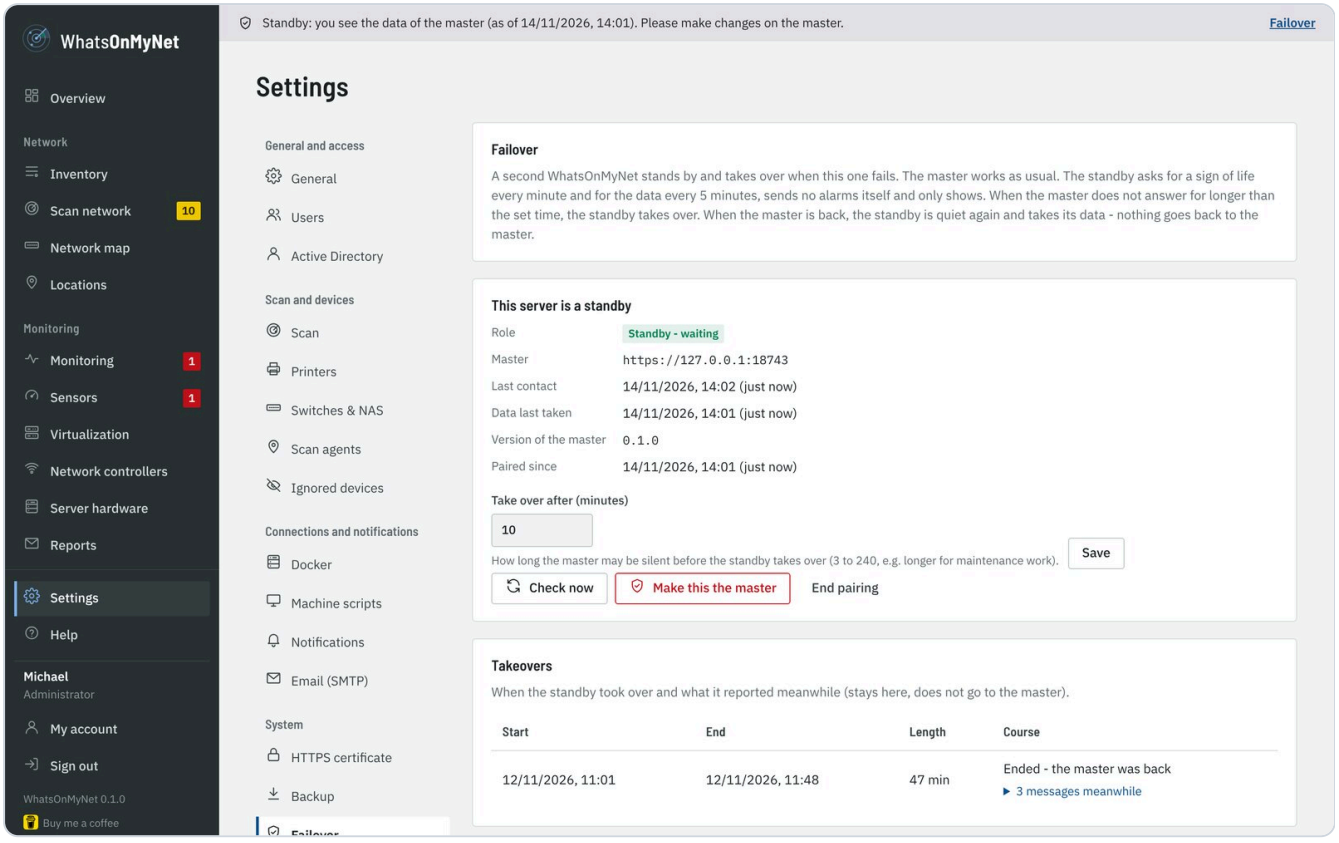
- **"The certificate of the DC is not trusted"**: *Fetch certificate from DC* and confirm.
- **"The name in the certificate does not fit"**: enter the name from the certificate as domain controller (fetching shows it). Only if you reach the DC by its IP address: switch *Check the server name in the certificate* off.
- **"Service account: sign-in refused"**: name or password of the service account wrong, or the account is locked.
- **"Group not found"**: check the spelling of the group in AD.

- **"No role"**: the user is in none of the groups entered.

Scan agent for branch offices

For a branch office a second container runs there as scan agent. Under *Settings → Scan agents* add an agent for the location; put the `.env` shown next to `docker-compose.agent.yml` from the release and start `docker compose -f docker-compose.agent.yml up -d`. The agent opens no port, fetches jobs over HTTPS and reports only addresses of its location.

Failover: master and standby



Failover: standby

A second WhatsOnMyNet (another machine, best on another host or in another room) stands by and takes over when the first one fails.

Setting up (once):

1. On the first server (**master**): *Settings → Failover → Make this the master*. A **pairing code** appears (`WOMN1-…`). It is shown only now and gives full access - treat it like a password.
2. On the second server (freshly installed or existing): *Settings → Failover → This server as standby*. Enter the address of the master (e.g. `10.0.10.5` or `womn1.firma.local:8743`) and the code, if you like the time until it takes over (10 minutes by default, 3 to 240) - **Pair**.

When pairing, the standby takes **all data of the master** (devices, monitors, users, channels, settings) and its key (`data/secret.key`) too, so it can read stored passwords. Its previous key is kept as `secret.key.before-standby`. If `WOMN_SECRET` is set on the standby, it must have the same value as on the master. Both servers need the same version.

In operation:

- The **master** works as always and does not have to do anything.
- The **standby** asks the master every minute and takes its data every 5 minutes. It checks nothing, scans nothing and sends no alarms. A note shows at the top; nothing can be changed there (please on the master).
- The connection uses HTTPS; the standby checks the fingerprint of the master's certificate from the code. If the key of the certificate stays the same on renewal (always with the self-signed one), nothing needs to be done. With a new certificate with a new key, create a new code on the master and pair again.

When the master fails: if it does not answer for longer than the set time, the **standby takes over**. It then checks and alarms itself and reports to all default channels: "Failover: the master ... has not answered for 10 min - this server (standby) has taken over monitoring." For longer maintenance on the master, raise the time on the standby beforehand.

When the master comes back: as soon as it answers, the standby is quiet again ("... is back") and takes its data. **Nothing goes back to the master.** What the standby reported meanwhile is listed on the standby under *Takeovers*.

When the master does not come back: on the standby, **Make this the master**. It is then the master for good and shows a new pairing code. If the old master comes up again after all, it learns this on its first contact and stops alarming. Best pair it there as standby of the new master.

Ending: *End pairing* on either side. On the master: the standby does not take over any more and stays quiet until decided there. On the standby: it works alone afterwards (if the master still runs, alarms arrive twice). A new pairing code on the master also lets the current standby go.

Backup and updates

- Once a day WhatsOnMyNet backs up the database to `data/backups/` (the first time one hour after the start); additionally before every database update.
- *Settings → Backup*: download a backup, back up now, restore a backup (the current state is backed up first, then the program restarts).
- **Keep `data/secret.key` (or `WOMN_SECRET`) as well** - without it stored passwords in a backup are unreadable.
- Updates: in Dockhand/Watchtower or `docker compose pull && docker compose up -d`. Once a month a security update with new packages is released.

What's new? A click on the version number at the bottom left of the menu shows *About WhatsOnMyNet*: the changes of the latest versions, the link to the source code, the website and a

coffee link. The coffee link is also right at the bottom of the menu. This works for every user, not only for administrators.

Troubleshooting

- **Interface not reachable**: open the host firewall for the HTTPS port. With host networking Docker/Dockhand shows no port mapping - that is correct.
- **No printer/switch data**: switch on SNMP at the device, check the community under *Settings → Scan*, "Query now" on the device page.
- **Locked out**: `docker compose exec whatsonmynet womn-admin reset <user>` sets a new password, makes the account administrator again and switches two-factor sign-in off. `womn-admin create <user>` adds a new administrator.
- **Network drive reports "protocol not supported"**: the device only speaks the old SMB1 - it is not used for security reasons. Switch on SMB2/3 at the device (Synology: *File Services → SMB → Advanced*).
- **Diagnostic package**: *Settings → Diagnostics* makes a ZIP with the log (kept for 30 days), warnings and errors, monitors and sensors with their checks, devices with the values their sensors see, hypervisors, scans, settings and the changes since the oldest version in the period - for the last 1 to 30 days or since the last update. Anonymised by default: names and addresses become placeholders (GERAET-001, 10.99.0.7 …) that stay the same in every package and can be looked up there. Passwords, keys and communities are never included; nothing is sent anywhere.
- **Log**: *Settings → Log* shows who changed what; technical messages are in the container log.